QueryAPIGate HOW-TO GUIDE 11 · QUERIES

Browse a connection's schema before writing a query

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=11-browse-a-connections-schema

---

Time: 5 minutes. **You'll end up with:** a quick way to see what tables/columns actually exist - and their types, nullability, primary and foreign keys - before guessing at column names in a query. Two ways to get there: the admin UI, or the same thing over the API.

In the admin UI: API Designer's Schema tab

Open **API Designer**, pick a connection, and click the **Schema** tab on the right side of the screen - next to **Recent Queries** and **Settings**. It has two sub-tabs:

- **Tables** - every table/view this connection can see.
- **Columns** - click a table in the Tables list and its columns appear here, each with its type, and a **PK** badge for a primary key or an **FK → table.column** badge showing exactly what a foreign key points to.

Click a column name to insert it at the cursor in the SQL editor - verified: with the editor containing `SELECT * FROM orders WHERE` and the cursor at the end, clicking `customer_id` in the Columns list turned it into `SELECT * FROM orders WHERE customer_id` - no copy-pasting column names by hand.

The same thing over the API

```shell
curl http://127.0.0.1:5000/connections/shop/schema -H 'X-API-Key: demo-key'
```

```json
{
  "tables": [
    {"name": "customers", "schema": "main", "type": "table", "columns": [
      {"name": "id", "type": "INTEGER", "nullable": true, "position": 1, "primary_key": true,
"foreign_key": null},
      {"name": "name", "type": "TEXT", "nullable": false, "position": 2, "primary_key": false,
"foreign_key": null}
    ]},
    {"name": "orders", "schema": "main", "type": "table", "columns": [
      {"name": "id", "type": "INTEGER", "nullable": true, "position": 1, "primary_key": true,
"foreign_key": null},
      {"name": "customer_id", "type": "INTEGER", "nullable": false, "position": 2, "primary_key": false,
       "foreign_key": {"table": "customers", "column": "id"}},
      {"name": "total", "type": "REAL", "nullable": true, "position": 3, "primary_key": false,
"foreign_key": null}
    ]}
  ],
  "truncated": false
}
```

Verified against a real SQLite schema with a real foreign key - the response above is the actual output, not a hand-written example. This is the exact same data the admin UI's Schema tab renders; there's no separate documentation or cache to keep in sync with the real database.

Primary and foreign keys are filled in on PostgreSQL, MySQL, SQLite and DuckDB; on the other types `primary_key` is always `false` and `foreign_key` always `null` - see the support matrix for what each type's schema browser covers.

`truncated: true` means there was more than `ROW_CAP` (5000) rows of catalogue data to show - large enough that no real schema should ever actually hit it, there purely to bound one request's memory use.

A Mongo connection's "schema" is just collection names, no columns - Mongo documents are schemaless, so there's nothing to introspect beyond what collections exist (see Connect to MongoDB).

Seeing a table's real `CREATE TABLE` text

```shell
curl 'http://127.0.0.1:5000/connections/shop/table_ddl?table=orders' -H 'X-API-Key: demo-key'
```

```json
{"ddl": "CREATE TABLE orders (id INTEGER PRIMARY KEY, customer_id INTEGER NOT NULL, total REAL, FOREIGN
KEY (customer_id) REFERENCES customers(id))"}
```

Verified real, exact DDL from SQLite's own catalogue. **Only MySQL, SQLite and ClickHouse support this** - Postgres, H2, generic `jdbc`, DuckDB and Mongo connections don't have an equivalent call built in. Asking for a table that doesn't exist on the connection is a clean 404, verified:

```shell
curl 'http://127.0.0.1:5000/connections/shop/table_ddl?table=nope' -H 'X-API-Key: demo-key'
# {"error": "'nope' is not a table on 'shop'"}
```

(Not "any string you send becomes a SQL identifier" - the table name is checked against the connection's real schema first; nothing you pass here ever reaches the database as unverified SQL.)

Browsing a different database on the same server

Both endpoints accept `?database=<name>` to look at a different database on the same connection's server (the one the admin UI's "Default database" / database picker uses) - but only the **admin key** may do this, verified:

```shell
curl 'http://127.0.0.1:5000/connections/shop/schema?database=other' -H 'X-API-Key: <a scoped key>'
# {"error": "Only the admin key may browse a different database on this connection"}
```

A scoped key is limited to exactly the database its connection was configured with; redirecting it at a sibling database the admin never explicitly granted isn't something a connection grant alone unlocks.

Next steps

- Turn your first SQL query into a REST API - the schema browser in context, writing a real query against what you just looked at.
- Connect a database - if you haven't got a connection to browse yet.